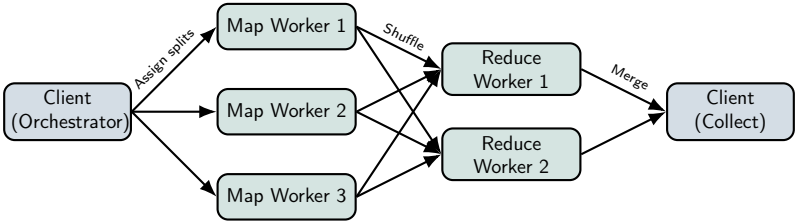
Distributed MapReduce on Lab Machines

Project presentation

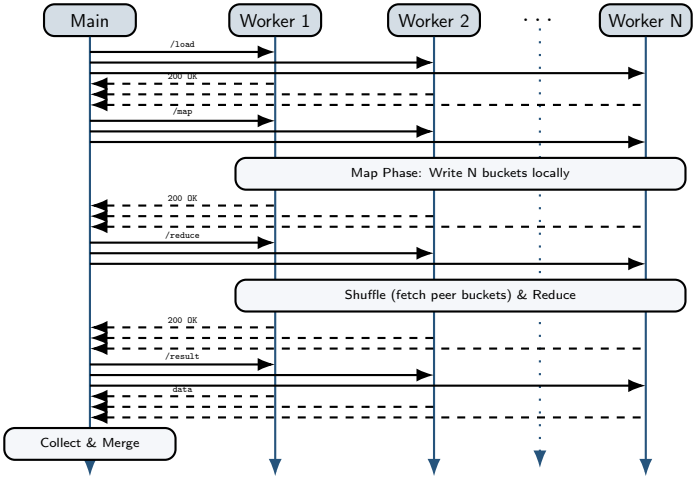
What We Built

- Automatic host discovery from `https://tp.telecom-paris.fr/ajax.php`
- Go orchestrator that builds, deploys, starts, monitors, and cleans workers
- HTTP worker API for `load -> map -> shuffle -> reduce -> result`
- Direct Common Crawl WET download from `data.commoncrawl.org`
- Four jobs: `wordcount`, `langdetect`, `domainpop`, `docdensity`

Architecture



Execution Sequence



Protocol and Dataflow

1. **Load:** Send $\leq$64MB chunks or WET URLs.
2. **Map:** Write `N` bucket files locally.
3. **Shuffle+reduce:** Fetch peer buckets via HTTP.
4. **Collect:** Merge & write result locally.

Map assignment: Orchestrator split + `POST /data`
Reducer assignment: `FNV-32a(key) % N`

Fault Tolerance and Operations

Failure Detection:

- A background health watcher polls `GET /health` every 5 seconds.
- Two consecutive failures mark a host as **dead**.

Recovery Execution (What happens when a node fails):

- The orchestrator pulls a **spare host** from the `hosts.txt` pool.
- It deploys a new worker binary over SSH to the spare.
- The system **replays** the `/load` and `/map` phases for the failed slot.
- If no spare host is available, the failed slot is reassigned to an existing healthy worker.
- Logical **slots** (`0..N-1`) stay fixed even if the physical host changes.

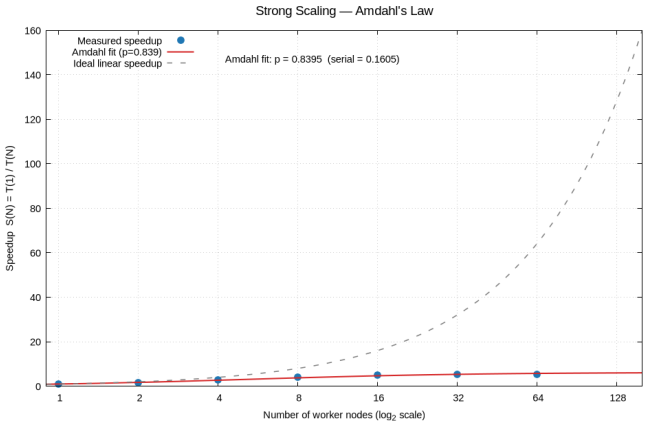
Use Cases Beyond Wordcount

Job	What it measures	Example output
`wordcount`	Token frequency	`the 239298`
`langdetect`	Language signal counts	`lang:en`, `lang:fr`
`domainpop`	Crawled domain popularity	`doi.org`, `youtube.com`
`docdensity`	Content size and density	`stat:lines`, `hist:len.*`

Executing a specific job

```
# The orchestrator injects the selected job into the worker binary
./mapreduce.sh -job scripts/jobs/langdetect \
  -commoncrawl \
  -crawl CC-MAIN-2026-21 \
  -n 8
```

Measurements and Amdahl's Law



Go Source Files (64 files):

- 1 `node`: 0.973 s (1.00x)
- 2 `nodes`: 0.494 s (1.97x)
- 4 `nodes`: 0.383 s (2.54x)
- 8 `nodes`: **0.266 s** (**3.66x**)
- 16 `nodes`: 1.412 s (0.69x)
- 64 `nodes`: 10.452 s (0.09x)

Methodology: Strong-scaling test on 64 Go files, varying active nodes from 1 to 64. Reference baseline is $N = 1$.

Three-Way Comparison

	Our MapReduce	**Hadoop**	**Kafka Streams**
Model	Batch, 1 stage	Batch, multi-stage	Stream, unbounded
Storage	`/tmp`, no replication	HDFS 3× replication	Replicated broker log
Data locality	No — HTTP pull	Yes — run where data lives	Broker partitions
Stragglers	Not handled	Speculative execution	N/A
8-node wordcount	0.27 s	not run	10.1 s

Biggest gaps vs. Hadoop

(1) HDFS data locality — we transfer all input at load time. (2) Speculative execution — one slow node blocks the whole phase.

Pain Points: What Broke and Why?

During development and testing at scale, we encountered major blockers:

1. **Zombie Processes:** Killed orchestrators left orphan worker processes running on remote lab machines, locking up assigned ports.
   - *Fix:* Made cleanup idempotent (matching port/process ID) and added health monitoring.

Optimization and Future Work

Performance Optimization & Profiling

- **In-Mapper Combiner:** Introduce an aggregation tree or combiner step in the mapper to drastically reduce HTTP requests and intermediate data size.
- **Streaming IO:** Stream intermediate bucket results instead of keeping maps fully in memory. This prevents OOM crashes observed with large Common Crawl files at N=1 and N=2.

Fault Tolerance Improvements

- **Durable State:** The current MapReduce state is volatile in `/tmp`. Implementing replicated bucket storage could provide stronger, Hadoop-like fault tolerance without full phase replays.

Demo and Discussion Plan

Live demo path

```
# Run word count on a local file (10k words) using 10 nodes
./mapreduce.sh -job scripts/jobs/wordcount \
  -input data/demo_text.txt -n 10 -output data/result.txt

# Fault Tolerance Demonstration:
# Kill a worker mid-computation to show orchestrator recovery
ssh tp-1a201-xx "pkill -9 worker"
```

Team Work Distribution

- **Guilherme:** Machine discovery, Deployment scripts, Fault tolerance, Amdahl experiments
- **Daniele:** Architecture, Protocol design
- **Marcin:** Kafka and comparison experiments